```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

# 1. Define the Objective / Fitness Function
def fitness_function(position, X, y):
    """
    Evaluates the quality of a selected feature subset.
    Returns a fitness score to be MINIMIZED.
    """
    # Convert continuous/probabilities to a binary mask
    # A feature is selected if its probability passes a threshold
    feature_mask = position > 0.5
    num_selected_features = np.sum(feature_mask)

    # If no features are selected, return a terrible fitness score
    if num_selected_features == 0:
        return 1.0

    # Select features
    X_selected = X[:, feature_mask]

    # Use cross-validation to get an accurate estimation of performance
    clf = RandomForestClassifier(n_estimators=50, random_state=42)
    scores = cross_val_score(clf, X_selected, y, cv=3, scoring='accuracy')
    error_rate = 1.0 - np.mean(scores)

    # Penalize large feature subsets (alpha balances accuracy vs. subset size)
    alpha = 0.95
    num_features = X.shape[1]
    feature_penalty = num_selected_features / num_features

    # Fitness formula to minimize
    fitness = alpha * error_rate + (1 - alpha) * feature_penalty
    return fitness

# 2. Define the Particle Structure
class Particle:
    def __init__(self, num_features):
        # Initialize random positions between 0 and 1
        self.position = np.random.uniform(0, 1, num_features)
        # Initialize random velocities
        self.velocity = np.random.uniform(-1, 1, num_features)

        # Track personal best
        self.best_position = np.copy(self.position)
        self.best_fitness = float('inf')

# 3. Main BPSO Optimization Loop
def binary_pso_feature_selection(X, y, num_particles=20, max_iter=15):
    num_features = X.shape[1]

    # Hyperparameters
    w = 0.729   # Inertia weight
    c1 = 1.494  # Cognitive coefficient (personal best influence)
    c2 = 1.494  # Social coefficient (global best influence)

    # Initialize Swarm
    swarm = [Particle(num_features) for _ in range(num_particles)]
    global_best_position = np.zeros(num_features)
    global_best_fitness = float('inf')

    # Optimization Loop
    for it in range(max_iter):
        for particle in swarm:
            # Evaluate fitness
            current_fitness = fitness_function(particle.position, X, y)

            # Update Personal Best
            if current_fitness < particle.best_fitness:
                particle.best_fitness = current_fitness
                particle.best_position = np.copy(particle.position)

            # Update Global Best
            if current_fitness < global_best_fitness:
                global_best_fitness = current_fitness
                global_best_position = np.copy(particle.position)

        # Update Velocities and Positions
        for particle in swarm:
```

```
        for particle in swarm:
            r1, r2 = np.random.rand(num_features), np.random.rand(num_features)

            # Velocity equation
            particle.velocity = (w * particle.velocity +
                                 c1 * r1 * (particle.best_position - particle.position) +
                                 c2 * r2 * (global_best_position - particle.position))

            # Clamp velocity to avoid overflow in sigmoid function
            particle.velocity = np.clip(particle.velocity, -6, 6)

            # Apply Sigmoid transformation to calculate selection probabilities
            sigmoid_v = 1 / (1 + np.exp(-particle.velocity))
            particle.position = sigmoid_v

        print(f"Iteration {it+1}/{max_iter} - Best Global Fitness: {global_best_fitness:.4f}")

    # Final chosen features
    selected_features_mask = global_best_position > 0.5
    return selected_features_mask

# 4. Run Execution Example
if __name__ == "__main__":
    # Load sample dataset (30 features)
    data = load_breast_cancer()
    X, y = data.data, data.target
    feature_names = data.feature_names

    print(f"Original dataset shape: {X.shape}")

    # Run optimizer
    best_mask = binary_pso_feature_selection(X, y, num_particles=15, max_iter=10)

    # Display final results
    chosen_indices = np.where(best_mask)[0]
    print("\n=== Feature Selection Optimization Complete ===")
    print(f"Original Number of Features: {X.shape[1]}")
    print(f"Reduced Number of Features: {len(chosen_indices)}")
    print(f"Selected Features: {list(feature_names[chosen_indices])}")
```

```
Original dataset shape: (569, 30)
Iteration 1/10 - Best Global Fitness: 0.0584
Iteration 2/10 - Best Global Fitness: 0.0584
Iteration 3/10 - Best Global Fitness: 0.0500
Iteration 4/10 - Best Global Fitness: 0.0500
Iteration 5/10 - Best Global Fitness: 0.0500
Iteration 6/10 - Best Global Fitness: 0.0500
Iteration 7/10 - Best Global Fitness: 0.0500
Iteration 8/10 - Best Global Fitness: 0.0500
Iteration 9/10 - Best Global Fitness: 0.0484
Iteration 10/10 - Best Global Fitness: 0.0484

=== Feature Selection Optimization Complete ===
Original Number of Features: 30
Reduced Number of Features: 10
Selected Features: [np.str_('mean concave points'), np.str_('radius error'), np.str_('texture error'), np.str_('perimeter er
```